

Plano de Ensino e Aprendizagem

1) Identificação

Curso	(417) Ciência da Computação - Manhã
Carga horária	80 h
Disciplina	(G03C3) Pesquisa e Ordenação
Semestre letivo	2026/2 - Graduação
Professor	Alexandre de Oliveira Zamberlan

2) Ementa

Métodos de ordenação; Pesquisa em memória principal; Pesquisa em memória secundária; Aplicação e comparação dos métodos em diferentes linguagens.

3) Objetivo da Disciplina – Competências

Unidade 1: Métodos de ordenação

Unidade 2: Pesquisa em memória principal

Unidade 3: Pesquisa em memória secundária

Unidade 4: Aplicação e comparação dos métodos em diferentes linguagens

4) Abertura da Disciplina

Na disciplina de Pesquisa e Ordenação estudaremos os principais algoritmos de ordenação e de pesquisa, a fim de agilizar os processos de busca, tanto em memória principal, quanto em secundária, sempre com foco em como aplicar tais algoritmos em situações problemas do dia-a-dia. Além disso, as soluções construídas serão via as linguagens C, C++, C#, Java e Python, com a utilização de boas práticas de desenvolvimento de sistemas em situações de problemas do mundo real.

5) Caracterização da metodologia de ensino

A disciplina será presencial em sala de aula, laboratório de informática e com apoio da ferramenta Minha UFN. Os conteúdos serão apresentados de forma expositivo-dialogados e com práticas nas linguagens C, C++, Python, Java e C#, instaladas nos computadores dos alunos e/ou nas máquinas do laboratório. O repositório GITHUB será usado para armazenar os programas, modelos, diagramas e anotações de aula desenvolvidos. Já a ferramenta Minha UFN funcionará como mecanismo de comunicação entre professor e aluno em aulas específicas a combinar.

6) Avaliação da aprendizagem

Serão critérios de avaliação:

- capacidade de contextualizar teoria à prática;
- construção de soluções diversificadas aos problemas propostos;
- envolvimento nos trabalhos;
- envolvimento nas aulas com participação crítica ao conteúdo e aos exercícios apresentados;
- funcionamento correto dos programas, além da qualidade da implementação e sua eficiência;
- provas e trabalhos: análise e discussão sobre soluções apresentadas;

vii) anotações diárias das aulas ou em caderno ou no GITHUB pessoal.

A nota será aferida por meio de três avaliações parciais.

As notas 1, 2 e 3 serão obtidas da seguinte forma:

$N1 = (\text{Participação efetiva } 0.2) + (\text{Produto de aprendizagem } 0.5) + (\text{Exercícios } 0.3)$

$N2 = (\text{Participação efetiva } 0.2) + (\text{Produto de aprendizagem } 0.5) + (\text{Exercícios } 0.3)$

$N3 = (\text{Participação efetiva } 0.2) + (\text{Produto de aprendizagem } 0.5) + (\text{Exercícios } 0.3)$

A nota final da disciplina será obtida por meio da média aritmética simples entre as notas N1, N2 e N3 conforme cálculo abaixo:

$\text{Nota Final} = (N1 + N2 + N3) / 3$

É possível que, durante o semestre, seja verificada a possibilidade de realizar exercícios ou trabalhos de recuperação. Neste caso, a nota do exercício/trabalho ajudará a compor uma nota parcial com a próxima prova escrita, o que será avisado aos alunos.

Para o aluno ser aprovado precisará de, no mínimo, 75% de frequência às aulas.

Aluno com média semestral igual ou superior a 6.0 estará aprovado, caso contrário, reprovado.

A participação em aula e anotações diárias são mecanismos de avaliação.

Serão aceitas somente as justificativas de ausências em avaliações teóricas previstas pela Central do Aluno e apresentadas até 48 horas após a ocorrência.

7) Bibliografia básica

CORMEN, Thomas H. **Algoritmos: teoria e prática**. Rio de Janeiro, RJ: Campus, 2002. 916 p.

GOODRICH, Michael T.; TAMASSIA, Roberto. **Estruturas de dados e algoritmos em java**. 2. ed. Porto Alegre: Bookman, 2002. 584 p.

ZIVIANI, Nivio. **Projeto de algoritmos com implementações em Pascal e C**. 5. ed. São Paulo, SP: Pioneira, 2002. 267 p.

8) Bibliografia complementar

AZEREDO, Paulo A. **Métodos de classificação de dados e análise de suas complexidades**. Rio de Janeiro, RJ: Campus, 1996. 132 p.

EDELWEISS, Nina; GALANTE, Renata. **Estruturas de dados**. Porto Alegre, RS: Bookman, 2009. 261 p. (Série livros didáticos informática ufrgs ; 18). ISBN 978-85-7780-381-1.

SANTOS, Clesio Saraiva dos; AZEREDO, Paulo Alberto de. **Tabelas: organização e pesquisa**. Porto Alegre: Sagra Luzzato, 2001. 85 p. (Livros Didáticos; 10).

SCHILDT, Herbert. **C completo e total**. 3. ed. rev. atual. São Paulo, SP: Makron Books, 2008. 827 p.

SZWARCITER, Jayme Luiz; MARKENZON, Lilian. **Estruturas de dados e seus algoritmos**. 2. ed. rev. 1994. Rio de Janeiro, RJ: LTC, 1994. 320 p.

Roteiro de estudos

1º aula / 1º semana

Modalidade da aula: Presencial - 27 de July de 2026

Unidade 1 - Métodos de ordenação Unidade 1 - Métodos de ordenação Unidade 1 - Métodos de ordenação Unidade 1 - Métodos de ordenação Unidade 1 - Métodos de ordenação Unidade 1 - Métodos de ordenação Unidade 2 - Métodos de pesquisa Unidade 2 - Métodos de pesquisa Unidade 2 - Métodos de pesquisa Unidade 2 - Métodos de pesquisa Unidade 3 - Pesquisa em memória secundária Unidade 3 - Pesquisa em memória secundária Unidade 3 - Pesquisa em memória secundária Unidade 4 - Aplicação e comparação dos métodos em diferentes linguagens Unidade 4 - Aplicação e comparação dos métodos em diferentes linguagens Unidade 4 - Aplicação e comparação dos métodos em diferentes linguagens

- Experiência formativa

Sobre o que refletiremos?

Como aplicar os métodos de ordenação em problemas reais? Como implementar tais métodos nas linguagens C, C++, Python, Java e C#? Como os métodos de ordenação são fundamentais em outras áreas da Computação?

Sobre o que refletiremos?

Como aplicar os métodos de ordenação em problemas reais? Como implementar tais métodos nas linguagens C, C++,

Python, Java e C#? Como os métodos de ordenação são fundamentais em outras áreas da Computação?

Sobre o que refletiremos?

Como aplicar os métodos de ordenação em problemas reais? Como implementar tais métodos nas linguagens C, C++, Python, Java e C#? Como os métodos de ordenação são fundamentais em outras áreas da Computação?

Sobre o que refletiremos?

Identificar, declarar, implementar e comparar, nas linguagens C++, Java e C# os métodos de ordenação agitação e pente.

Sobre o que refletiremos?

Identificar, declarar, implementar e comparar, nas linguagens C++, Java e C# o método Quick.

Sobre o que refletiremos?

Identificar, declarar, implementar e comparar, nas linguagens C++, Java e C# os métodos Bucket e Radix.

Sobre o que refletiremos?

Identificar, declarar, implementar e comparar, nas linguagens C++, Java e C# o método Heap.

Sobre o que refletiremos?

Pesquisa sequencial e binária em memória principal

Sobre o que refletiremos?

A implementação de métodos de pesquisa e compreensão dos diferentes desempenhos em algoritmos.

Sobre o que refletiremos?

Armazenamento e busca por complexidade $O(1)$, via funções e tabelas Hash.

Sobre o que refletiremos?

Tabelas Hash em diferentes linguagens de programação.

Sobre o que refletiremos?

Como aplicar os métodos de pesquisa em memória secundária em problemas reais? Como implementar tais métodos nas linguagens C, C++, Java e C#? Como os métodos de pesquisa secundária são fundamentais em outras áreas da Computação?

Sobre o que refletiremos?

O uso de técnicas de balanceamento em árvores para agilizar a busca.

Sobre o que refletiremos?

As diferentes técnicas para armazenar e buscar dados volumosos em memória secundária e como isso reflete no desempenho de sistemas computacionais.

Sobre o que refletiremos?

Os algoritmos são independentes de linguagens? As linguagens atuais suportam tais algoritmos?

Sobre o que refletiremos?

A gestão de dados vindo de memória secundária.

Sobre o que refletiremos?

Fechamento da disciplina.

- Guia de Aprendizagem

Nesta aula vamos discutir o Plano de Ensino, a importância da ordenação, conceitos fundamentais e as categorias de ordenação. Além disso, vamos implementar os métodos seleção, inserção e bolha sempre com temporizador e contadores de comparações e trocas.

Nesta aula vamos implementar os métodos seleção, inserção e bolha sempre com temporizador e contadores de comparações e trocas, nas linguagens de programação C++, Java, Python e C#.

Nesta aula vamos implementar os métodos agitação e pente sempre com temporizador e contadores de comparações e trocas, nas linguagens de programação C++, Java e C#.

Nesta aula vamos implementar os métodos agitação e pente sempre com temporizador e contadores de comparações e trocas, nas linguagens de programação C++, Java e C#.

Nesta aula vamos implementar o método Quick, sempre com temporizador e contadores de comparações e trocas, nas linguagens de programação C++, Java e C#.

Nesta aula vamos implementar os métodos Bucket e Radix, sempre com temporizador e contadores de comparações e trocas, nas linguagens de programação C++, Java e C#.

Nesta aula vamos implementar o método Heap, sempre com temporizador e contadores de comparações e trocas, nas linguagens de programação C++, Java e C#.

Nesta aula vamos discutir e reavaliar o sistema construído no processo de avaliação.

Nesta aula vamos discutir a importância dos algoritmos de pesquisa em sistemas computacionais. Além disso, vamos

implementar os métodos de pesquisa sequencial e binária sempre com temporizador e contadores de comparações.

Nesta aula vamos discutir a importância dos algoritmos de pesquisa em sistemas computacionais. Além disso, vamos implementar os métodos de pesquisa sequencial e binária sempre com temporizador e contadores de comparações.

Nesta aula vamos discutir a importância de Tabelas Hash em sistemas computacionais. Além disso, vamos implementar as estruturas aberta e fechada de Tabela e seus métodos de endereçamento.

Nesta aula vamos aplicar o uso de Hashing Table em C# e Java.

Nesta aula vamos discutir a importância dos algoritmos de pesquisa em memória secundária dentro de sistemas computacionais. Além disso, vamos implementar árvores DSW e AVL.

Nesta aula vamos discutir a importância dos algoritmos de pesquisa em memória secundária dentro de sistemas computacionais. Além disso, vamos implementar árvores DSW e AVL.

Nesta aula vamos discutir a importância de Árvores B em memória secundária dentro de sistemas computacionais. Além disso, vamos simular o uso de árvores B em sistemas computacionais.

Nesta aula vamos implementar Árvores B em memória usando C# ou Java.

Nesta aula vamos implementar Árvores B em memória usando C# ou Java.

Fechamento da disciplina

- Encaminhamento próxima aula - Atividade Discente Efetiva

Pesquise sobre o ambiente GitHub (versionamento de código), crie conta, clone o repositório da disciplina, crie seu repositório pessoal da disciplina.

A partir dos códigos exemplos do GitHub da disciplina, responda a lista de exercícios. Lembre de colocar seus códigos no seu repositório da disciplina.

A partir dos códigos exemplos do GitHub da disciplina, responda a lista de exercícios. Lembre de colocar seus códigos no seu repositório da disciplina.

A partir dos códigos exemplos do GitHub da disciplina, responda a lista de exercícios. Lembre de colocar seus códigos no seu repositório da disciplina.

A partir dos códigos exemplos do GitHub da disciplina, responda a lista de exercícios. Lembre de colocar seus códigos no seu repositório da disciplina.

A partir dos códigos exemplos do GitHub da disciplina, responda a lista de exercícios. Lembre de colocar seus códigos no seu repositório da disciplina.

A partir dos códigos exemplos do GitHub da disciplina, responda a lista de exercícios. Lembre de colocar seus códigos no seu repositório da disciplina.

Corrija os erros e/ou equívocos existentes no código pessoal da avaliação.

A partir dos códigos exemplos no Github da disciplina, complemente-os e os adicione em seus GitHubs pessoais.

A partir dos códigos exemplos no Github da disciplina, complemente-os e os adicione em seus GitHubs pessoais.

A partir dos códigos exemplos no Github da disciplina, complemente-os e os adicione em seus GitHubs pessoais.

A partir dos códigos exemplos no Github da disciplina, complemente-os e os adicione em seus GitHubs pessoais.

A partir dos códigos exemplos no Github da disciplina, complemente-os e os adicione em seus GitHubs pessoais.

A partir dos códigos exemplos no Github da disciplina, complemente-os e os adicione em seus GitHubs pessoais.

Veja o simulador e/ou vídeo fornecido pelo professor.

Veja o simulador e/ou vídeo fornecido pelo professor.

Não haverá.

Não há

- Referência Bibliográfica

CORMEN, Thomas H et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002.

CORMEN, Thomas H et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002.

CORMEN, Thomas H et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002.

CORMEN, Thomas H et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002.

CORMEN, Thomas H et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002.

CORMEN, Thomas H et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002.

CORMEN, Thomas H et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002.

Github da disciplina.

CORMEN, Thomas H et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002.

CORMEN, Thomas H et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002.

CORMEN, Thomas H et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002.

CORMEN, Thomas H et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002.

CORMEN, Thomas H et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002.

CORMEN, Thomas H et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002.

Veja o simulador e/ou vídeo fornecido pelo professor.

Veja o simulador e/ou vídeo fornecido pelo professor.

Simulador fornecido pelo professor.

Não há

- Introdução

Fazer um programa que trate todos os algoritmos estudados. O programa deve carregar números de arquivo e aplicar os métodos, temporizando-os e contabilizando comparações e trocas.

Fazer um programa que implemente um sistema para armazenar nomes de pessoas em uma estrutura Hash. O programa deve permitir que o usuário Cadastre, Pesquise, Liste e Exclua nomes de pessoas. Entretanto, o armazenamento deve ser via uma estrutura Hash com Tratamento de Colisão por Lista Encadeada.

- Repertório profissional

O programa deve ser construído no editor de codificação de preferência e ao final deve ser disponibilizado no GitHub pessoal do aluno.

O programa deve ser construído no editor de codificação de preferência e ao final deve ser disponibilizado no GitHub pessoal do aluno.

- Recursos e ferramentas

Linguagens de programação trabalhadas no curso, bem como seus respectivos IDE.

- Produto de aprendizagem

Implementar um programa, em modo gráfico, orientado a objetos que aplique e compare todos os métodos de ordenação e de pesquisa estudados em sala de aula.

Implementar um programa, em modo gráfico, orientado a objetos que aplique e compare todos os métodos de ordenação e de pesquisa estudados em sala de aula.

- Atividade

Enviar o link do seu github pessoal